APUNTES

UD01 — Fundamentos del sistema operativo

Módulo: Sistemas Operativos Monopuesto · 1º SMR

Introducción

Antes de instalar un sistema operativo, configurarlo o administrarlo, necesitas entender qué es y cómo está organizado por dentro. Esta unidad te da el vocabulario y los conceptos base que vas a usar constantemente a lo largo de todo el curso: cuando dentro de unas semanas instales tu primera máquina virtual, cuando particiones un disco, cuando ejecutes tu primer comando `chmod`, vas a estar aplicando directamente lo que aprendas aquí.

Dos contenidos de esta unidad merecen especial atención porque los vas a usar de forma literal y constante a partir de ahora: **el sistema de archivos** (cómo se organiza la información en un disco) y **los permisos** (quién puede hacer qué con cada archivo). Por eso ocupan más de la mitad del peso de esta unidad — no porque sean "más difíciles", sino porque son los que de verdad vas a necesitar desde el primer día de prácticas con máquinas virtuales.

Al terminar esta unidad debes ser capaz de:

- Explicar qué es un sistema informático y dónde encaja el sistema operativo dentro de él.
- Describir qué hace un sistema operativo y cómo se organiza por capas.
- Distinguir un programa de un proceso, y nombrar los estados básicos de un proceso.
- Moverte conceptualmente por la jerarquía de archivos de Windows y de Linux.
- Convertir entre binario, octal y decimal para leer y escribir permisos, y calcular múltiplos del byte.
- Interpretar y asignar permisos de archivos y directorios.
- Explicar por qué existen los sistemas de archivos transaccionales.

1. El sistema informático y el software

¿Qué es un sistema informático?

Un **sistema informático** es la combinación de tres elementos que se necesitan mutuamente: **hardware** (los componentes físicos), **software** (los programas que hacen que ese hardware sirva para algo) y **usuarios** (personas u otros sistemas que lo utilizan). Ninguno de los tres funciona de forma útil por sí solo: el hardware sin software es inerte —como un coche sin conductor ni gasolina—, y el software sin hardware no tiene dónde ejecutarse.

Software de base y software de aplicación

Dentro del software, hay una distinción fundamental:

- **Software de base**: el que permite que el resto del sistema funcione. Incluye el sistema operativo, los controladores (drivers) y ciertas utilidades del sistema (antivirus, gestores de particiones...). Existe para que las aplicaciones no tengan que "hablar" directamente con el hardware.
- **Software de aplicación**: los programas que usa el usuario para realizar tareas concretas — un procesador de textos, un navegador, un videojuego.

Esta distinción no siempre es perfectamente nítida (por ejemplo, un antivirus se percibe como una aplicación más, pero funciona a bajo nivel dando soporte al resto del sistema), pero como regla general: si un programa existe para que el sistema funcione, es software de base; si existe para que el usuario haga algo concreto, es software de aplicación.

Niveles del sistema informático

Todo sistema informático se puede representar como una serie de capas, de más cercana al hardware a más cercana al usuario:

```
Usuario
    ↑
Software de aplicación
    ↑
Lenguajes / entornos de programación
    ↑
Software de base (sistema operativo, drivers)
    ↑
Hardware
```

Cada capa se apoya en la anterior y ofrece una base más sencilla de usar a la siguiente. El hardware es lo único que existe físicamente; todo lo demás son distintos niveles de software que hacen que ese hardware sea utilizable.

¿Dónde encajan los lenguajes de programación?

Entre el software de base y el software de aplicación existe otra capa: los **lenguajes de programación**, con los que se construye el software de aplicación (y también parte del software de base). Existen distintos niveles de lenguaje:

- **Lenguaje máquina**: instrucciones binarias que la CPU ejecuta directamente. Es el único lenguaje que el hardware entiende de verdad.
- **Ensamblador**: una representación algo más legible del lenguaje máquina, con una correspondencia casi directa entre instrucción y operación de la CPU.
- **Lenguajes de alto nivel** (Python, Java, C++...): mucho más cercanos a cómo razona una persona, pero que necesitan traducirse (mediante un compilador o un intérprete) antes de convertirse en instrucciones que la CPU pueda ejecutar.

No hace falta que sepas programar para esta unidad — lo importante es que entiendas que cuando alguien programa una aplicación, ese código termina bajando, nivel a nivel, hasta convertirse en instrucciones que la CPU ejecuta directamente.

Para practicar

Actividad: clasifica los siguientes elementos como hardware, software de base o software de aplicación, y justifica tu respuesta en una frase: el núcleo de Linux, un navegador web, el controlador de una tarjeta gráfica, una memoria RAM, un editor de código, un antivirus, el firmware de la BIOS/UEFI.

Ejemplo resuelto: *el controlador de una tarjeta gráfica* → software de base, porque no es una herramienta que el usuario utilice directamente para una tarea, sino una pieza de software que permite que el sistema operativo se comunique con ese componente de hardware concreto.

Resuelve el resto por tu cuenta, siguiendo el mismo razonamiento.

VAMOS A PRACTICAR: LOCALIZA SOFTWARE DE BASE REAL EN TU PROPIO EQUIPO

En Windows, abre el **Administrador de dispositivos** (búscalo en el menú Inicio) y despliega al menos tres categorías (por ejemplo, "Adaptadores de red", "Controladoras de sonido, vídeo y juegos" o "Adaptadores de pantalla"). Elige tres dispositivos y anota, para cada uno, el nombre del controlador que aparece en sus propiedades (pestaña "Controlador" → "Proveedor del controlador" y "Fecha del controlador").

Si tu equipo tiene también Linux instalado, compara con el comando `lspci` (dispositivos conectados a la placa base) o `lsusb` (dispositivos USB) ejecutado en una terminal: obtendrás una lista equivalente, aunque el sistema no use la palabra "controlador" de la misma forma explícita.

Reflexiona: ¿alguno de esos tres dispositivos dejaría de funcionar correctamente si le faltara su controlador, aunque el hardware siga físicamente bien conectado?

2. Funciones y arquitectura del sistema operativo

Las cuatro grandes funciones del sistema operativo

El sistema operativo cumple, de forma general, cuatro funciones:

- **Gestión de procesos**: reparte el tiempo de CPU entre los distintos programas en ejecución.
- **Gestión de memoria**: organiza la memoria RAM para que las aplicaciones no interfieran entre sí.
- **Gestión de archivos**: organiza y da acceso ordenado a la información guardada en el almacenamiento.
- **Gestión de entrada/salida**: media entre las aplicaciones y los periféricos (teclado, ratón, impresora, red...).

En esta unidad no vas a ver *cómo* decide el sistema operativo, por ejemplo, qué proceso ejecutar en cada instante (eso son los algoritmos de planificación) ni el detalle interno de cómo gestiona

la memoria — eso queda fuera del alcance de este curso. Lo importante aquí es que entiendas *qué* problema resuelve cada función.

Cómo gestionan la memoria los sistemas actuales

Los programas que tienes abiertos a la vez, sumados, suelen necesitar más memoria RAM de la que tiene instalada tu equipo — y aun así puedes tener docenas de aplicaciones abiertas sin que el sistema se caiga. La técnica que lo permite, usada tanto en **Windows 11** como en **Linux** actuales, se llama **paginación por demanda**.

En vez de cargar un programa entero en RAM de golpe, el sistema operativo lo divide en bloques (páginas) y **solo carga en RAM las páginas que se están usando en cada momento**. Si hace falta una página que no está cargada, el sistema la trae desde el almacenamiento; si hace falta sitio en RAM, saca de ahí alguna página que no se esté usando, guardándola temporalmente en disco (lo que Windows llama "archivo de paginación" y Linux llama "swap").

Esta es la razón real de por qué se pueden tener más aplicaciones abiertas de las que "caben" en la RAM instalada, y también de por qué el ordenador se ralentiza mucho cuando empieza a depender demasiado del disco para esto: el disco, incluso un SSD, es mucho más lento que la RAM.

VAMOS A PRACTICAR: COMPRUEBA LA MEMORIA REAL DE TU EQUIPO

Abre el Administrador de tareas de Windows (`Ctrl+Shift+Esc`) → pestaña **Rendimiento** → **Memoria** (en Linux, el Monitor del sistema, o el comando `free -h` en una terminal). Anota tres datos: la RAM total instalada, la RAM en uso ahora mismo, y si aparece alguna referencia a "memoria virtual"/"archivo de paginación" (Windows) o "swap" (Linux), y cuánto espacio tiene reservado.

Reflexiona: si cerraras ahora mismo todas las aplicaciones que tienes abiertas, ¿bajaría a cero el uso de RAM que muestra el sistema? ¿Por qué no?

Arquitectura del sistema operativo por capas

El sistema operativo, a su vez, se organiza también por capas:

```
Aplicaciones
    ↑
Shell (interfaz de usuario)
    ↑
Controladores (drivers)
    ↑
Núcleo (kernel)
    ↑
Hardware
```

- **Núcleo (kernel)**: la capa más interna, la única que se comunica directamente con el hardware. Gestiona procesos, memoria y dispositivos a bajo nivel.

- **Controladores (drivers)**: piezas de software, normalmente suministradas por el fabricante de cada componente, que permiten al núcleo comunicarse con un dispositivo concreto.
- **Shell**: la capa que permite al usuario dar instrucciones al sistema. Puede ser en **texto** (una línea de comandos, como bash o PowerShell) o en **gráficos** (un escritorio con ventanas e iconos).
- **Aplicaciones**: los programas que usa el usuario, apoyados en todas las capas anteriores.

¿Y la interfaz gráfica?

Es habitual pensar que la interfaz gráfica (el escritorio, las ventanas, los iconos) *es* el sistema operativo — sobre todo si nunca has usado un ordenador de otra forma. Pero no es así: la interfaz gráfica es simplemente **la versión gráfica de la capa de shell**, no una capa nueva ni el sistema operativo en sí.

La prueba más clara es que un sistema operativo puede funcionar perfectamente **sin ninguna interfaz gráfica**: es lo habitual en los servidores, que se administran por red y no necesitan que nadie se siente delante con teclado y ratón. El sistema operativo sigue estando completo — núcleo, gestión de procesos, memoria, archivos— aunque le falte la capa de shell en su versión gráfica.

Modo usuario y modo núcleo: quién puede tocar el hardware

En tu ordenador se están ejecutando ahora mismo decenas de programas a la vez, y todos comparten la misma CPU, la misma memoria RAM y los mismos dispositivos. Imagina qué pasaría si cualquiera de ellos pudiera hacer lo que quisiera: un programa con un fallo podría escribir encima de la memoria de otro programa (o del propio sistema operativo), un programa malicioso podría leer directamente el disco saltándose los permisos de los archivos, y un bucle mal programado podría quedarse con la CPU para siempre sin dejar funcionar a nadie más.

El sistema operativo no puede evitarlo solo con software: cuando un programa está ejecutándose, sus instrucciones van directamente a la CPU, sin pedir permiso a nadie. Por eso la protección la pone **el propio hardware**. Todos los procesadores actuales pueden trabajar en, al menos, dos **modos de ejecución**, y en cada momento saben en cuál están:

- **Modo núcleo** (*kernel mode*, también llamado **modo privilegiado** o modo supervisor): se puede ejecutar cualquier instrucción del procesador y acceder a toda la memoria y a todos los dispositivos. En este modo trabaja el **núcleo** del sistema operativo y, en Windows y Linux, también los **controladores**.
- **Modo usuario** (*user mode*): solo se permiten las instrucciones "normales" (cálculos, comparaciones, leer y escribir en la memoria propia...). Las instrucciones delicadas — acceder directamente a un dispositivo, cambiar la configuración de la memoria, desactivar interrupciones— están prohibidas. En este modo se ejecutan **las aplicaciones**: el navegador, el procesador de textos, un juego, la propia terminal...

	Modo núcleo	Modo usuario
¿Quién se ejecuta así?	Núcleo del SO y controladores	Aplicaciones y la mayoría de servicios

	Modo núcleo	Modo usuario
¿Qué puede hacer?	Todo: cualquier instrucción, toda la memoria, todos los dispositivos	Solo instrucciones normales y solo en su propia memoria
¿Qué pasa si intenta algo prohibido?	— (no hay nada prohibido)	La CPU lo bloquea y avisa al núcleo, que normalmente cierra ese programa
¿Qué pasa si tiene un fallo grave?	Cae todo el sistema: **pantallazo azul** en Windows, ***kernel panic*** en Linux	Se cierra solo ese programa ("la aplicación ha dejado de funcionar"); el resto sigue

Esta última fila es la que más vas a notar en la práctica: cuando se cierra de golpe una aplicación, el ordenador sigue funcionando; cuando falla un controlador, suele caer el sistema entero. La diferencia está en el modo en que se ejecutaba cada uno.

Detalle técnico: en los procesadores Intel/AMD (arquitectura x86) existen en realidad cuatro niveles de privilegio, llamados **anillos** (*rings*) y numerados del 0 al 3. Windows y Linux solo usan dos: el **anillo 0** para el modo núcleo y el **anillo 3** para el modo usuario. Por eso es habitual oír "esto se ejecuta en *ring 0*" como sinónimo de "se ejecuta en modo núcleo".

Las llamadas al sistema: pedir las cosas por ventanilla

Si una aplicación no puede tocar el disco, ¿cómo guarda un documento? **Se lo pide al núcleo.** Para eso existen las **llamadas al sistema** (*system calls*): una forma controlada de que un programa en modo usuario solicite un servicio al sistema operativo (abrir un archivo, escribir en él, enviar datos por la red, crear un proceso...).

Funciona como la ventanilla de un banco: el cliente no puede entrar en la cámara acorazada a coger su dinero, sino que lo pide en la ventanilla; el empleado comprueba que la petición es correcta (que la cuenta es suya, que tiene saldo) y es él quien entra a buscarlo. De la misma forma, cuando un programa hace una llamada al sistema:

1. El procesador **cambia a modo núcleo** y salta a un punto de entrada fijo del núcleo. El programa no puede elegir a qué parte del núcleo salta: solo "llama a la ventanilla".
2. El núcleo **comprueba** la petición: que el archivo existe, que ese usuario tiene permiso para escribir en él, que los datos pertenecen al propio programa...
3. Si todo es correcto, el núcleo hace el trabajo (por ejemplo, ordena al controlador del disco que escriba los datos).
4. El procesador **vuelve a modo usuario** y el programa recibe el resultado.

Los programadores no suelen hacer estas llamadas directamente, sino a través de bibliotecas del sistema (en Windows, la API de Windows; en Linux, la biblioteca estándar de C, *glibc*), pero por debajo siempre acaban en una llamada al sistema.

Además de las llamadas al sistema, el procesador pasa a modo núcleo en otros dos casos: cuando un programa provoca un **error** (dividir entre cero, acceder a memoria que no es suya, necesitar una página que no está en la RAM, como viste en la paginación por demanda) y cuando llega una **interrupción** del hardware (se pulsa una tecla, llega un paquete de red o salta el temporizador

del sistema). Gracias a ese temporizador, el núcleo recupera el control cada pocos milisegundos, y por eso ningún programa puede quedarse con la CPU para siempre, aunque tenga un bucle infinito.

Fíjate en el reparto de tareas que hay detrás de todo esto: **el sistema operativo decide, pero necesita la ayuda del procesador para hacer cumplir sus decisiones**. Es el sistema operativo quien decide qué proceso se ejecuta, cuánta memoria recibe cada uno o qué hacer cuando un programa comete un error; pero es el procesador quien, con el temporizador, le devuelve el control; quien, con la protección de memoria, impide que un proceso invada a otro; y quien, con el modo núcleo, impide que los programas se salten las reglas. De hecho, la CPU ni siquiera sabe qué es un "proceso": eso es una idea del sistema operativo, que es quien lleva la cuenta de cada uno.

Cambiar de modo **no es gratis**: el procesador tiene que guardar su estado, saltar al núcleo y volver, y cada cambio consume un pequeño tiempo. Un programa que hace millones de llamadas al sistema lo nota. Este coste es precisamente la clave para entender los distintos tipos de núcleo del apartado siguiente.

Cuidado con esta confusión: ser administrador NO es ejecutarse en modo núcleo. Cuando ejecutas un programa como administrador en Windows, o un comando con `sudo` en Linux, ese programa sigue ejecutándose en **modo usuario**. Ser administrador (o `root`) es un permiso **del sistema operativo**: el núcleo acepta más peticiones de ese programa. El modo núcleo es un estado **del procesador**. La relación entre ambos es que el administrador puede **instalar código que se ejecutará en modo núcleo**, como un controlador — por eso instalar drivers exige privilegios de administrador, y por eso hay que tener mucho cuidado con qué drivers se instalan.

VAMOS A PRACTICAR: ¿CUÁNTO TRABAJA EL NÚCLEO EN TU EQUIPO?

1. En Windows, abre el Administrador de tareas (`Ctrl+Shift+Esc`) → pestaña **Rendimiento** → **CPU**. Haz clic derecho sobre la gráfica y activa **"Mostrar tiempos de kernel"**. Aparece una segunda zona, más oscura: es el tiempo que la CPU pasa en **modo núcleo**; el resto es tiempo en modo usuario.
2. Con la gráfica a la vista, haz dos cosas distintas y observa qué zona sube en cada caso: primero, copia una carpeta grande de un sitio a otro (o descarga un archivo grande); después, reproduce un vídeo o abre una página web pesada.
3. (Opcional, en Linux) En una terminal, ejecuta `top` y fíjate en la línea `%Cpu(s)`: el valor `us` (*user*) es el tiempo en modo usuario y `sy` (*system*) el tiempo en modo núcleo. Pulsa `q` para salir.

Reflexiona: ¿en cuál de las dos tareas del paso 2 ha subido más el tiempo de kernel? ¿Por qué crees que copiar archivos obliga a trabajar más al núcleo? (Pista: ¿quién es el único que puede hablar con el disco?)

Tipos de núcleo

Con los dos modos de ejecución claros, ya se puede entender en qué se diferencian los tipos de núcleo. La pregunta clave es: **¿qué partes del sistema operativo se ejecutan en modo núcleo, y cuáles en modo usuario?** Existen tres modelos:

- **Monolítico**: todos los servicios básicos (gestión de procesos, memoria, sistemas de archivos, red, controladores...) se ejecutan juntos en **modo núcleo**. Como todo está "en el mismo sitio", las distintas partes se comunican directamente entre sí, sin cambios de modo, y eso lo hace **muy rápido**. El inconveniente es que un fallo en cualquier parte —por ejemplo, en un controlador— puede tumbar el sistema entero. Es el caso de **Linux**. Linux es además **modular**: puede cargar y descargar partes de su funcionalidad en caliente mediante **módulos** (normalmente controladores). Pero ojo: un módulo, una vez cargado, se ejecuta **dentro del núcleo, en modo núcleo**; "modular" no significa "aislado".
- **Microkernel**: en modo núcleo se deja solo lo mínimo imprescindible (comunicación entre procesos, reparto básico de la CPU y de la memoria). El resto de servicios —controladores, sistemas de archivos, red— se ejecutan como **procesos independientes en modo usuario**. Si un controlador falla, se cierra solo ese proceso y se puede volver a arrancar sin que caiga el sistema: es **más robusto**. A cambio, cualquier operación exige muchos mensajes entre procesos y muchos cambios de modo, y eso lo hace **más lento**. Se usa donde la fiabilidad importa más que el último punto de rendimiento: por ejemplo, **QNX**, muy presente en los sistemas de los coches, o **MINIX**.
- **Híbrido**: el diseño está organizado como un microkernel (en piezas separadas y bien definidas), pero, **por rendimiento**, la mayoría de esas piezas se ejecutan en modo núcleo, como en un monolítico. Es el caso de **Windows NT** (la base de todos los Windows modernos), donde los controladores y hasta parte del sistema gráfico se ejecutan en modo núcleo, y también de **macOS/iOS** (con su núcleo XNU). En la práctica, en cuanto a qué se ejecuta en cada modo, un núcleo híbrido se parece bastante más a un monolítico que a un microkernel.

	Monolítico	Microkernel	Híbrido
Qué hay en modo núcleo	Todo	Solo lo mínimo	Casi todo
Controladores	Modo núcleo	Modo usuario	Modo núcleo (mayoritariamente)
Rendimiento	Alto	Menor	Alto
Si falla un controlador	Puede caer todo el sistema	Se reinicia ese proceso	Puede caer todo el sistema
Ejemplos	Linux	QNX, MINIX	Windows NT, macOS (XNU)

No existe una jerarquía de calidad entre estos tres modelos: son decisiones de diseño distintas, cada una con sus ventajas e inconvenientes. Que Linux use un núcleo monolítico no lo hace "anticuado" frente a un microkernel, ni un núcleo híbrido es automáticamente "lo mejor de ambos mundos" sin matices.

Actualidad: en julio de 2024, una actualización defectuosa de un programa de seguridad llamado CrowdStrike Falcon provocó pantallazos azules en unos 8,5 millones de ordenadores con Windows en todo el mundo, parando aeropuertos, bancos y hospitales. El motivo de que el fallo fuera tan grave es que ese programa incluía un **controlador que se ejecutaba en modo núcleo**: el mismo error en un programa en modo usuario solo habría cerrado ese programa. Desde entonces, Microsoft trabaja para que los antivirus y programas de seguridad puedan funcionar **fuera** del modo núcleo.

Windows y Linux, capa a capa

Elemento	Windows	Linux
Núcleo	Núcleo NT (híbrido)	Núcleo Linux (monolítico modular)
Shell en texto	CMD / PowerShell	bash / zsh
Shell gráfico	Explorador de Windows	GNOME / KDE, entre otros
Gestión de software	Instaladores .exe/.msi, Microsoft Store, winget	Gestores de paquetes (APT, DNF)
Controladores	Suministrados por el fabricante, vía Windows Update o instalación manual	Suministrados por el fabricante o integrados en el propio núcleo

VAMOS A PRACTICAR: EXPLORA CAPA A CAPA EN TU PROPIO EQUIPO

Vas a comprobar, uno a uno, los elementos de la tabla anterior en el sistema que tengas delante:

1. **Núcleo.** En Windows, ejecuta `winver` (Inicio → escribe "winver") para ver la versión del núcleo NT. En Linux, ejecuta `uname -r` en una terminal.
2. **Shell en texto.** Abre una terminal (PowerShell/CMD en Windows, o una terminal en Linux) y ejecuta un comando sencillo, por ejemplo `dir` o `ls`.
3. **Shell gráfico.** Observa el propio escritorio que tienes delante — Explorador de Windows, o GNOME/KDE en Linux — e identifica que es, técnicamente, otra forma de shell, no una capa distinta.
4. **Gestión de software.** En Windows, abre la Microsoft Store; en Linux, ejecuta `apt list --installed | wc -l` (o abre el gestor gráfico de paquetes) para ver cuántos paquetes tienes instalados.
5. **Controladores.** Si ya hiciste la actividad del apartado 1 (Administrador de dispositivos), recupérala; si no, ábrelo ahora.

Reflexiona: ¿has usado las cinco capas para completar esta actividad, o solo alguna? ¿En cuál pasas más tiempo tú, en tu día a día como usuario?

Para practicar

Actividad: para cada una de estas situaciones, indica en qué capa de la arquitectura del sistema operativo se soluciona el problema (núcleo, controladores, shell, aplicaciones): "el ordenador no reconoce una impresora nueva", "quieres escribir un comando para renombrar 200 archivos a la vez", "una aplicación se cierra sola de forma repetida", "el sistema reparte la CPU entre varios programas abiertos a la vez".

Ejemplo resuelto: *"el ordenador no reconoce una impresora nueva"* → controladores: el sistema necesita el driver adecuado para poder comunicarse con ese dispositivo concreto.

Resuelve el resto por tu cuenta.

3. Procesos y sus estados

Programa y proceso no son lo mismo

Un **programa** es código almacenado —un archivo en disco, sin ejecutar—. Un **proceso** es ese programa **en ejecución**, con recursos ya asignados por el sistema operativo (memoria, tiempo de CPU). La diferencia importa: puedes tener un único archivo de programa (por ejemplo, un navegador) y sin embargo varios procesos de ese mismo programa ejecutándose a la vez.

Estados de un proceso

A lo largo de su vida, un proceso pasa por distintos estados:

- **Nuevo**: se acaba de crear.
- **Listo**: está preparado para ejecutarse, esperando su turno de CPU.
- **En ejecución**: la CPU está procesando sus instrucciones en ese instante.
- **Bloqueado**: está esperando algún recurso externo (datos del disco, respuesta de la red...) y no puede avanzar hasta que llegue. Es un estado normal y temporal, no un fallo.
- **Terminado**: ha finalizado su ejecución.

En esta unidad no vas a estudiar cómo decide el sistema operativo qué proceso ejecutar en cada momento —eso se conoce como planificación de procesos y queda fuera del alcance de este curso—, pero sí vas a retomar este contenido más adelante, cuando trabajes con herramientas reales como el Administrador de tareas de Windows o el comando `top` de Linux, que muestran estos procesos y estados en vivo.

Para practicar

Actividad: describe, usando los estados vistos, por qué pasa un proceso de navegador cuando abres una página web: desde que abres el navegador hasta que la página termina de cargar.

Ejemplo resuelto (primera parte): al abrir el navegador, el proceso pasa de **nuevo** a **listo**, y en cuanto la CPU le asigna tiempo, pasa a **ejecución**.

Continúa tú la secuencia hasta que la página termina de cargar, identificando en qué momento el proceso pasa a estar **bloqueado** y por qué.

VAMOS A PRACTICAR: UN PROGRAMA, VARIOS PROCESOS

Abre tu navegador habitual y crea tres pestañas nuevas, cada una en una página distinta. Después abre el Administrador de tareas de Windows (o ejecuta `ps aux | grep <nombre-del-navegador>` en una terminal Linux) y cuenta cuántos procesos aparecen asociados a ese navegador.

Reflexiona: si el navegador es un único programa instalado en tu disco, ¿por qué aparecen varios procesos al ejecutarlo? ¿Qué le pasaría al resto de pestañas si uno solo de esos procesos se bloqueara?

4. Sistema de archivos: organización y atributos

¿Para qué sirve un sistema de archivos?

Cuando guardas un archivo, no se coloca en un lugar aleatorio del disco: el **sistema de archivos** es la estructura que organiza y localiza toda la información almacenada. Sin él, un disco sería simplemente un espacio de bytes sin ningún orden ni forma de encontrar nada.

La jerarquía de directorios en Windows

Windows organiza los discos con **letras de unidad** (`C:\`, `D:\`...), cada una con su propia estructura de carpetas. Dentro de la unidad principal encontrarás carpetas típicas como `Users` (usuarios), `Program Files` (programas instalados) o `Windows`. Las rutas se escriben separando carpetas con la barra invertida `\`, y pueden ser:

- **Absolutas**: desde la raíz de la unidad, por ejemplo `C:\Users\Alejandro\Documents\informe.docx`.
- **Relativas**: desde la carpeta en la que ya estás, por ejemplo `Documents\informe.docx` si ya estás dentro de `C:\Users\Alejandro`.

Ojo: el Explorador "traduce" algunos nombres. En un Windows en español, el Explorador muestra carpetas como `Usuarios`, `Documentos` o `Archivos de programa`, pero en el disco esas carpetas se llaman en realidad `Users`, `Documents` y `Program Files`. El nombre en español es solo una etiqueta para mostrar. Lo notarás en cuanto hagas clic en la barra de direcciones para ver la ruta como texto, o cuando trabajes en una terminal: la ruta real es `C:\Users\...`, y es la que tienes que escribir.

La jerarquía de directorios en Linux

Linux no usa letras de unidad: existe una **única raíz** (`/`), y cualquier disco o partición adicional se **monta** dentro de ese árbol único, apareciendo como una carpeta más — no como una letra nueva. Algunos de los directorios principales son:

- `/home` — carpetas personales de cada usuario.
- `/etc` — archivos de configuración del sistema.
- `/var` — datos que cambian con el tiempo (registros, colas de impresión...).
- `/bin` y `/usr` — programas del sistema.

Las rutas en Linux se separan con la barra `/` (no `\`), y también pueden ser absolutas o relativas, igual que en Windows.

Subir de nivel en una ruta relativa: `.` y `..`

Hasta ahora, las rutas relativas que has visto siempre "bajaban" desde la carpeta en la que estás hacia una carpeta que está dentro de ella. Pero a menudo el archivo que buscas no está por debajo de ti, sino en una carpeta "hermana" o más arriba en el árbol. Para eso, todo directorio contiene dos nombres especiales, que existen tanto en Linux como en Windows:

- `.` (un punto): el propio directorio en el que estás.
- `..` (dos puntos): el **directorio padre**, es decir, el que contiene al directorio en el que estás. Usarlo en una ruta equivale a "subir un nivel".

Fíjate en este árbol:

```
/
└── home/
    └── alumno/
        ├── documentos/
        │   └── informe.txt
        └── practicas/
            └── ud01/
```

Si estás en `/home/alumno/practicas` y quieres llegar a `informe.txt`, no puedes bajar directamente: `informe.txt` no está dentro de `practicas`. Primero tienes que subir a `alumno` y desde ahí bajar a `documentos`. La ruta relativa es `../documentos/informe.txt`, que se lee paso a paso así:

1. `..` → subes de `/home/alumno/practicas` a `/home/alumno`.
2. `documentos` → bajas a `/home/alumno/documentos`.
3. `informe.txt` → llegas al archivo.

Puedes encadenar varios `..` para subir más de un nivel: desde `/home/alumno/practicas/ud01`, la misma ruta sería `../../documentos/informe.txt` (subes dos niveles, hasta `alumno`, y luego bajas). En Windows funciona exactamente igual, cambiando el separador: `..\Documents\informe.docx`.

Dos ideas para no equivocarte: una ruta relativa **nunca empieza por `/`** en Linux (ni por una letra de unidad en Windows), porque entonces sería absoluta; y el resultado de una ruta relativa **depende de dónde estés**, mientras que una absoluta lleva siempre al mismo sitio. Cuando empieces a usar la terminal de Linux, escribirás `cd ..` constantemente para subir de carpeta: es exactamente este mismo `...`.

La diferencia que más suele fallar: mayúsculas y minúsculas

Windows no distingue mayúsculas de minúsculas en los nombres de archivo: `Documento.txt` y `documento.txt` son el mismo archivo. **Linux sí distingue**: esos mismos dos nombres serían dos archivos completamente distintos. Esta diferencia es la causa más habitual de que "un comando no funcione" cuando se empieza a trabajar con la terminal de Linux — conviene tenerla muy presente desde ya, antes de que te encuentres con ese problema en la práctica.

Atributos de archivos y directorios

Tanto Windows como Linux permiten marcar archivos y carpetas con **atributos** que indican propiedades adicionales:

- **En Windows**: solo lectura, oculto, sistema, archivo listo para archivar. Se consultan y modifican desde las propiedades del archivo en el Explorador.
- **En Linux**: los archivos ocultos se marcan poniendo un punto al principio del nombre (`.archivo`). Existen otros atributos más avanzados gestionables con la herramienta `chattr`, que verás con más detalle en unidades posteriores.

Un archivo oculto **no está protegido** de ningún modo especial: simplemente no se muestra por defecto. No confundas "oculto" con "seguro".

Para practicar

Actividad (rutas): dado el siguiente fragmento de árbol de directorios Linux, escribe la ruta absoluta y una ruta relativa posible (suponiendo que partes de `/home/alumno`) hasta el archivo `informe.txt`:

```
/
├── home/
│   └── alumno/
│       └── documentos/
│           └── informe.txt
├── etc/
└── var/
```

Ejemplo resuelto: ruta absoluta → `/home/alumno/documentos/informe.txt`. Ruta relativa desde `/home/alumno` → `documentos/informe.txt`.

Actividad (subir de nivel): con ese mismo árbol, escribe una ruta relativa hasta `informe.txt` suponiendo que partes de `/etc`. Pista: primero tienes que subir hasta la raíz.

Actividad (mayúsculas): de estos tres nombres de archivo — `Informe.docx`, `informe.docx`, `INFORME.docx` —, ¿cuántos archivos distintos son en Windows? ¿Y en Linux? Justifica tu respuesta con lo aprendido en este apartado.

VAMOS A PRACTICAR: COMPRUÉBALO EN LOS DOS SISTEMAS DE TU PROPIO EQUIPO

Los equipos del aula tienen Windows y Linux disponibles, así que puedes comprobar esto con datos reales en vez de solo razonarlo:

1. En Windows, crea un archivo de texto llamado `prueba.txt` en una carpeta cualquiera. En esa misma carpeta, intenta crear ahora otro archivo llamado `Prueba.txt`. ¿Qué pasa?
2. Arranca el mismo equipo en Linux y repite exactamente el mismo experimento: crea `prueba.txt` y, en la misma carpeta, `Prueba.txt`. ¿Cuántos archivos distintos tienes ahora?
3. Comprueba también, en Windows, si puedes tener un archivo llamado `PRUEBA.txt` "encima" del que ya tenías, y anota qué mensaje te muestra el sistema si lo intentas.

Anota lo que observas en cada sistema y relaciónalo con lo explicado en este apartado — no hay truco: es exactamente el comportamiento que se acaba de describir, comprobado de primera mano.

Vamos a practicar: tu propia ruta real

Abre el explorador de archivos de tu equipo Windows y navega hasta tu carpeta personal de Documentos. Haz clic en la barra de direcciones para que se muestre la ruta completa como texto y cópiala. Escríbela aquí como ruta absoluta y, después, escribe una ruta relativa hasta esa misma carpeta partiendo de tu carpeta de usuario (`C:\Users\<tu usuario>`). Como cada alumno tiene un nombre de usuario distinto, tu ruta será distinta a la de cualquier compañero — no la puedes copiar de nadie.

Vamos a practicar: oculta un archivo de verdad

Crea un archivo de prueba en tu equipo y márcalo como oculto: en Windows, clic derecho → Propiedades → activa la casilla "Oculto"; en Linux, basta con renombrarlo añadiendo un punto delante del nombre (`.prueba.txt`). Comprueba que desaparece del explorador de archivos con la vista de "elementos ocultos" desactivada, y que vuelve a verse al activarla.

Reflexiona: ¿has protegido realmente el contenido de ese archivo con esto? ¿Qué pasaría si alguien activa la opción de "mostrar archivos ocultos"?

5. Binario y octal aplicados a permisos, codificación de caracteres y unidades de medida de la información

Por qué el ordenador usa binario

Un transistor solo tiene dos estados posibles: encendido o apagado. Por eso los ordenadores representan toda la información en **binario** (base 2, con solo dos dígitos: 0 y 1) — no es una elección arbitraria, es una consecuencia directa de cómo funciona el hardware.

Conversión entre decimal y binario

Cualquier número decimal se puede representar en binario. Por ejemplo, el número decimal 13 se descompone como potencias de 2:

$$13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0 \Rightarrow 1101 \text{ en binario}$$

Y al revés, para convertir un binario a decimal, se suman las potencias de 2 correspondientes a cada posición con un 1: `1101` → 1×8 + 1×4 + 0×2 + 1×1 = 13.

En esta unidad trabajarás con números de hasta un byte (8 bits, valores de 0 a 255) — lo suficiente para lo que necesitas: leer y escribir permisos.

Codificación de caracteres: de binario a texto

Un byte no solo sirve para representar cantidades: también puede representar un **carácter** de texto, según qué tabla de correspondencia se use.

ASCII, el primer sistema de codificación ampliamente usado, asigna un valor numérico (de 0 a 127) a cada letra del alfabeto inglés, los dígitos y algunos símbolos. Es suficiente para escribir en inglés, pero **no incluye acentos, eñes, ni alfabetos distintos del latino** — se quedó corto en cuanto la informática se extendió más allá del mundo anglosajón.

Tabla ASCII reducida

No hace falta memorizarla: consúltala cada vez que necesites codificar o decodificar un carácter en las actividades de esta unidad.

Letras mayúsculas

Letra	Dec	Letra	Dec	Letra	Dec	Letra	Dec
A	65	H	72	O	79	V	86
B	66	I	73	P	80	W	87
C	67	J	74	Q	81	X	88
D	68	K	75	R	82	Y	89
E	69	L	76	S	83	Z	90
F	70	M	77	T	84		
G	71	N	78	U	85		

Letras minúsculas

Letra	Dec	Letra	Dec	Letra	Dec	Letra	Dec
a	97	h	104	o	111	v	118
b	98	i	105	p	112	w	119
c	99	j	106	q	113	x	120
d	100	k	107	r	114	y	121
e	101	l	108	s	115	z	122
f	102	m	109	t	116		

Letra	Dec	Letra	Dec	Letra	Dec	Letra	Dec
g	103	n	110	u	117		

Dígitos y símbolos habituales

Carácter	Dec	Carácter	Dec	Carácter	Dec	Carácter	Dec
(espacio)	32	3	51	6	54	9	57
0	48	4	52	7	55	.	46
1	49	5	53	8	56	,	44
2	50						

Unicode resuelve esa limitación: es una tabla enorme que asigna a cada carácter posible un número único, llamado ***code point***, que se escribe como U+XXXX en hexadecimal. Por ejemplo: 'A' → U+0041, 'ñ' → U+00F1, '€' → U+20AC, '😀' → U+1F600. El estándar actual define posiciones hasta U+10FFFF (más de 1,1 millones posibles), y se diseñó para ser compatible con ASCII: sus primeros 128 caracteres coinciden exactamente.

Es importante distinguir dos cosas que suelen confundirse: **Unicode solo dice *qué número* le corresponde a cada carácter — no dice *cómo* guardar ese número como bytes**. Eso es trabajo de una **codificación** (encoding): UTF-8, UTF-16 o UTF-32.

UTF-8: cómo un programa convierte bytes en caracteres

UTF-8 es, con diferencia, la codificación más usada hoy: se estima que la usan más del 97-98% de las páginas web, y es la codificación por defecto en Linux, en la mayoría de lenguajes de programación y en formatos como JSON.

UTF-8 codifica cada carácter usando entre **1 y 4 bytes**, según lo alto que sea su *code point*:

Rango de *code point*	Bytes usados	Patrón de bits
U+0000 – U+007F (ASCII)	1 byte	`0xxxxxxx`
U+0080 – U+07FF	2 bytes	`110xxxxx 10xxxxxx`
U+0800 – U+FFFF	3 bytes	`1110xxxx 10xxxxxx 10xxxxxx`
U+10000 – U+10FFFF	4 bytes	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

Para qué sirven los bits fijos.

Cuando un programa lee un archivo de texto, no recibe "caracteres": recibe una fila de bytes seguidos, sin ningún separador entre ellos. Si todos los caracteres ocuparan 1 byte, como en ASCII, no habría problema; pero en UTF-8 unos ocupan 1 byte y otros 2, 3 o 4. Así que el programa necesita poder averiguar, mirando solo los bits de cada byte, si ese byte **empieza** un carácter o es la **continuación** de uno anterior, y cuántos bytes ocupa cada carácter. Esa información la llevan escrita los bits fijos del principio de cada byte:

El byte empieza por…	Significa
`0`	Carácter completo de 1 byte (un carácter ASCII)

El byte empieza por…	Significa
`110`	Empieza un carácter de **2** bytes
`1110`	Empieza un carácter de **3** bytes
`11110`	Empieza un carácter de **4** bytes
`10`	No empieza nada: es la **continuación** de un carácter anterior

Fíjate en el truco: el número de `1` con que empieza el primer byte coincide con el número de bytes del carácter. Con esta tabla, un programa lee sin ninguna duda esta fila de bytes:

```
01001000  → empieza por 0   → carácter de 1 byte: 'H'
11000011  → empieza por 110 → carácter de 2 bytes: este y el siguiente
10110001  → empieza por 10  → continuación del anterior → juntos forman 'ñ'
01101111  → empieza por 0   → carácter de 1 byte: 'o'
```

El resultado es el texto "Hño". (Cómo se obtienen exactamente los bytes de la 'ñ' lo verás en el ejemplo resuelto de más abajo.)

¿Por qué no aprovechar todos los bits de los bytes de continuación? Porque entonces un byte de continuación podría tener cualquier valor y se confundiría con otra cosa:

- **Con un carácter ASCII.** Si la segunda mitad de una 'ñ' pudiera empezar por `0`, un programa que busca, por ejemplo, la barra `/` de una ruta podría encontrar una "barra" falsa a mitad de la 'ñ' y cortar ahí el nombre del archivo. En UTF-8 eso es imposible: ningún byte de un carácter de varios bytes empieza por `0`.
- **Con el principio de otro carácter.** Si una continuación pudiera empezar por `110`, el programa no sabría si es el final de la 'ñ' o el comienzo de un carácter nuevo.
- **Con todo lo que viene después, si hay un error.** Si se pierde un byte (una descarga cortada, un archivo dañado) o un programa empieza a leer a mitad del texto, le basta con saltar los bytes que empiezan por `10` hasta encontrar uno que no lo haga, y desde ahí vuelve a leer bien. Se estropea solo el carácter dañado, no el resto del texto.

El precio es que se "pierden" algunos bits: un carácter de 2 bytes tiene 16 bits, pero solo 11 sirven para guardar su número; los otros 5 son marcas. Los diseñadores de UTF-8 aceptaron ese coste a cambio de la seguridad que dan esas marcas, y como el texto más habitual es ASCII, que ocupa 1 byte sin ninguna marca extra, en la práctica el coste es pequeño.

Por qué UTF-8 es compatible con ASCII: cualquier carácter ASCII (0-127) se codifica en UTF-8 con un único byte, con el mismo valor que ya tenía en ASCII. Es decir, **cualquier texto ASCII ya es, sin cambiar ni un bit, un texto UTF-8 válido** — la razón principal de que UTF-8 se haya impuesto sin romper nada de lo que ya existía.

Ejemplo resuelto: codificar 'ñ' en UTF-8

1. El *code point* de 'ñ' es U+00F1 = 241 en decimal.
2. 241 está entre 128 y 2047 → necesita 2 bytes, patrón `110xxxxx 10xxxxxx` (11 bits disponibles).
3. 241 en binario, con 11 bits: `00011110001`.

4. Reparto: los primeros 5 bits (`00011`) van al primer byte; los últimos 6 (`110001`), al segundo.
5. Resultado: `11000011 10110001` → en hexadecimal, **C3 B1**. Así se guarda exactamente la letra 'ñ' en cualquier archivo de texto en UTF-8.

UTF-16: dónde sigue vivo aunque no se vea

UTF-16 usa bloques de 2 bytes para la mayoría de caracteres, y "pares subrogados" (dos bloques de 2 bytes combinados) para los que quedan fuera de ese rango, como muchos emojis. En la web ha perdido terreno frente a UTF-8, pero sigue siendo la codificación interna que usan **Windows** (en sus APIs internas), **Java** (en su tipo `String`) y **JavaScript** (en la representación interna de las cadenas de texto).

Cuando un archivo de texto se abre "lleno de símbolos raros" en vez de mostrar tildes o eñes correctamente, casi siempre es un problema de codificación: los bytes son los mismos, pero se están interpretando con el patrón de bits equivocado, así que el programa reconstruye un carácter distinto al que realmente se guardó.

VAMOS A PRACTICAR: COMPRUEBA TÚ MISMO QUE UTF-8 NO USA SIEMPRE EL MISMO NÚMERO DE BYTES

Como cada carácter puede ocupar de 1 a 4 bytes, un archivo de texto no pesa "número de caracteres × un tamaño fijo" — pesa la suma de lo que ocupa cada carácter concreto.

1. Calcula a mano, carácter a carácter, cuántos bytes ocupará el texto `Año` 😀 guardado en UTF-8 (usa la tabla de patrones de bits: recuerda que 'ñ' ya la tienes resuelta, y que un emoji cae muy por encima de U+FFFF).
2. Abre un editor de **texto plano** — Bloc de notas en Windows, o un editor de texto en Linux (no uses Word ni ningún procesador de textos: añaden formato y no son texto plano). Escribe exactamente ese texto y guárdalo asegurándote de que la codificación sea UTF-8 (en Bloc de notas, se elige en el desplegable "Codificación" del cuadro de guardar).
3. Comprueba el tamaño del archivo en bytes: en Windows, clic derecho → Propiedades (o el comando `dir` en una terminal); en Linux, `ls -l archivo.txt` o `wc -c archivo.txt`.
4. ¿Coincide con tu cálculo? Si el tamaño real es un poco mayor de lo esperado, no te preocupes: puede deberse a un carácter de salto de línea añadido automáticamente al guardar (1-2 bytes más), o a que el editor haya incluido una marca **BOM** al principio del archivo (3 bytes más) — infórmate de qué es un BOM si te encuentras con esta diferencia.

Ejemplo resuelto (cálculo): `A` (1 byte) + `ñ` (2 bytes) + `o` (1 byte) + espacio (1 byte) + 😀 (4 bytes) = **9 bytes para 5 caracteres**.

PARA SABER MÁS (NO ENTRA EN EL EXAMEN): UN EMOJI QUE OCUPA 20 BYTES

En clase comprobamos que el emoji de una persona en silla de ruedas mirando a la derecha, 🧑‍🦽‍➡️, ocupa **20 bytes**. ¿Cómo es posible, si UTF-8 usa como mucho 4 bytes por carácter? Porque ese símbolo **no es un carácter: son seis**, que el sistema dibuja como si fueran uno solo:

Code point	Qué es	Bytes en UTF-8
U+1F9D1	🧑 persona	4
U+200D	ZWJ, el "pegamento" invisible	3
U+1F9BD	🦽 silla de ruedas manual	4
U+200D	ZWJ, el "pegamento" invisible	3
U+27A1	➡️ flecha hacia la derecha	3
U+FE0F	indica "dibújalo como emoji"	3
	Total	**20**

La pieza clave es el **ZWJ** (*Zero Width Joiner*, "unión de anchura cero"): un carácter invisible que le dice al sistema "si sabes dibujar esta combinación como un único símbolo, hazlo". Si el sistema conoce la combinación, ves un solo emoji; si no la conoce (por ejemplo, un sistema antiguo), ves las piezas por separado: 🧑🦽➡️. De hecho, según el programa con el que estés leyendo estos apuntes, puede que en este recuadro veas el emoji partido en sus piezas: es justo lo que se está explicando. Así se construyen muchos emojis: las familias, los tonos de piel, las profesiones o las banderas. La versión sin flecha, 🧑‍🦽, ocupa 11 bytes; la flecha "invisible" añade 9.

Esto demuestra que hay tres cosas distintas que solemos llamar "carácter":

Nivel	Qué es	En este emoji
Lo que ves en pantalla	El símbolo	1
Code points	Los "caracteres" de Unicode	6
Bytes	Lo que ocupa guardado en UTF-8	20

Cuando los apuntes dicen que UTF-8 usa "de 1 a 4 bytes por carácter", se refieren a cada **code point**. Y tiene consecuencias prácticas: un campo de un formulario que admite "como máximo 10 caracteres" puede estar contando bytes, code points o símbolos, según cómo se haya programado. Por eso algunas webs rechazan un nombre con un emoji, o lo cortan por la mitad y dejan un símbolo roto.

Sistema octal: por qué nos interesa aquí

El sistema **octal** (base 8) agrupa los bits de tres en tres para representarlos con un solo dígito (0-7). No es casualidad que se use precisamente para permisos: **un permiso `rwx` son exactamente**

3 bits (uno por cada tipo de permiso, activado o desactivado), así que **un dígito octal representa un conjunto completo de permisos**. Es la razón real de que se use octal aquí, y no otra base.

Sistema hexadecimal: por qué existe y dónde lo vas a ver

El sistema **hexadecimal** (base 16, dígitos del 0 al 9 y luego de la A a la F para el 10-15) agrupa el binario en bloques de **4 bits**, no de 3 como el octal.

Un byte son 8 bits. 8 no se divide limpiamente en grupos de 3 (sobra resto), pero sí exactamente en dos grupos de 4 — así que **un byte completo se escribe siempre con exactamente 2 dígitos hexadecimales**. Por eso el hexadecimal es la notación estándar para cualquier dato organizado en bytes: colores web (`#RRGGBB`, un byte por cada componente de color), direcciones MAC de tarjetas de red (6 bytes = 12 dígitos hex, agrupados como `00:1A:2B:3C:4D:5E`) y, de hecho, **los bytes de la codificación UTF-8 que ya has visto** (`C3 B1`, `C3 A9`...): ahora ya sabes qué es esa notación.

Binario	Hex	Binario	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

Ejemplo resuelto: convertir el byte 201 (decimal) a hexadecimal. 201 en binario (8 bits) es `11001001`. Se separa en dos grupos de 4: `1100` y `1001`. Con la tabla: `1100` = C, `1001` = 9. Resultado: **C9**.

Unidades de medida de la información

El byte es la unidad base para medir cantidades de información, pero sus múltiplos se definen de **dos formas distintas**, según dos normas diferentes:

- El **Sistema Internacional (SI)**: kilo, mega, giga... en **base 10** (potencias de 1000). Es el que usan los fabricantes de discos para anunciar su capacidad: 1 kB = 1000 bytes, 1 MB = 1000 kB, 1 GB = 1000 MB.
- La norma **IEC 80000-13**: kibi, mebi, gibi... en **base 2** (potencias de 1024). Es la que suelen calcular internamente los sistemas operativos: 1 KiB = 1024 bytes, 1 MiB = 1024 KiB, 1 GiB = 1024 MiB.

El problema es que, por costumbre histórica, la mayoría de sistemas operativos calculan en GiB pero lo etiquetan en pantalla como "GB" — lo cual genera una confusión muy conocida.

Tabla completa de múltiplos

Kilo/mega/giga son solo los tres primeros escalones; la tabla continúa mucho más allá, y sigue exactamente el mismo patrón en las dos normas:

Prefijo SI	Símbolo	Potencia de 10	Prefijo IEC	Símbolo	Potencia de 2
kilo	k	10^3	kibi	Ki	2^{10}
mega	M	10^6	mebi	Mi	2^{20}
giga	G	10^9	gibi	Gi	2^{30}
tera	T	10^{12}	tebi	Ti	2^{40}
peta	P	10^{15}	pebi	Pi	2^{50}
exa	E	10^{18}	exbi	Ei	2^{60}
zetta	Z	10^{21}	zebi	Zi	2^{70}
yotta	Y	10^{24}	yobi	Yi	2^{80}

En el día a día solo vas a manejar hasta giga/tera (discos, memorias) o como mucho peta (centros de datos grandes). Exa, zetta y yotta existen y se usan —por ejemplo, se estima que el tráfico mundial de datos ronda ya varios zettabytes al año—, pero no son magnitudes con las que trabajará un técnico de sistemas en su día a día.

Bits y bytes: por qué 1 Gbps no son 1 GB de descarga

Hasta ahora hemos medido cantidades de información en **bytes** (1 byte = 8 bits). Pero hay un contexto donde la unidad habitual no es el byte, sino el **bit**: la velocidad de transmisión de datos en redes.

- La **capacidad de almacenamiento** (discos, memorias, archivos) se mide en bytes: KB, MB, GB, TB...
- La **velocidad de conexión** (internet, redes) se mide en bits por segundo: kbps, Mbps, Gbps...

La convención de notación distingue ambas cosas por la caja de la letra: **b minúscula = bit, B mayúscula = byte**. No es un detalle tipográfico sin importancia: **Gb** (gigabit) y **GB** (gigabyte) difieren en un factor de 8.

Esto explica una confusión muy habitual: una conexión a internet de "300 Mbps" no descarga a 300 megabytes por segundo, sino a 300 megabits por segundo — que son 300 ÷ 8 = 37,5 MB/s como máximo teórico (la velocidad real suele ser aún menor, por la sobrecarga del propio protocolo de red).

El caso real: el disco de 500 GB que se queda en ~465 GB

Un disco anunciado como "de 500 GB" tiene exactamente $500 \times 10^9 = 500.000.000.000$ bytes — ese es el Sistema Internacional, tal y como lo usa el fabricante, y no hay ningún engaño en ello. Cuando el sistema operativo muestra la capacidad de ese mismo disco, la calcula en GiB (base 2), aunque la etiquete como "GB":

$$500.000.000.000 \div 1024^3 = 500.000.000.000 \div 1.073.741.824 \approx 465,7 \text{ GiB}$$

No se ha perdido ningún byte: son exactamente los mismos, contados con dos reglas de agrupación distintas. Es importante que entiendas este cálculo, porque en cuanto empieces a particionar discos para tus máquinas virtuales vas a comprobarlo con tus propios ojos.

VAMOS A PRACTICAR: COMPRUÉBALO CON TU PROPIO DISCO

En Windows, abre el Explorador de archivos, haz clic derecho sobre la unidad `C:` y entra en Propiedades. Anota la capacidad total que muestra el sistema (en GB). Después, busca la capacidad anunciada por el fabricante para ese mismo disco (en la caja, la factura o las especificaciones técnicas del equipo). ¿Coinciden exactamente los dos números? Si no, calcula a qué se debe la diferencia usando lo aprendido en este apartado.

Si no tienes a mano el dato del fabricante, comprueba al menos que el número que Windows llama "GB" es en realidad GiB: en la misma ventana de Propiedades suele mostrarse también la capacidad exacta en bytes; divide ese valor entre 1024^3 y comprueba que el resultado coincide con el "GB" que muestra el sistema.

Permisos de archivos y directorios

En Linux, cada archivo o directorio tiene permisos definidos para **tres conjuntos de usuarios**: el **propietario**, el **grupo** al que pertenece, y **otros** (el resto de usuarios del sistema). Para cada conjunto existen tres tipos de permiso:

- **r** (read / lectura): poder ver el contenido.
- **w** (write / escritura): poder modificar el contenido.
- **x** (execute / ejecución): poder ejecutar el archivo si es un programa o script; en un directorio, significa poder **entrar** en él y acceder a su contenido (no "ejecutarlo").

Estos permisos se representan de dos formas equivalentes:

- **Simbólica**: por ejemplo `rwxr-xr--` — el propietario tiene lectura/escritura/ejecución, el grupo tiene lectura/ejecución, y el resto solo lectura.
- **Numérica (octal)**: la misma información, un dígito por conjunto de usuarios. `rwxr-xr--` equivale a **754**.

En **Windows**, el sistema de archivos NTFS también gestiona permisos, pero con un modelo distinto y más granular (listas de control de acceso, o ACL), que no se resume con esta misma notación numérica. Lo verás en detalle en una unidad posterior.

De binario a permiso, paso a paso

Para pasar de un valor octal a lo que puede hacer cada usuario, conviene descomponerlo en binario. Por ejemplo, el permiso **640**:

- 6 → `110` → lectura y escritura, sin ejecución (propietario).
- 4 → `100` → solo lectura (grupo).
- 0 → `000` → ningún permiso (resto de usuarios).

Es decir, `640` equivale a `rw-r-----`.

Para practicar

Actividad (ASCII): a partir de la tabla ASCII reducida que se te proporcione, traduce una palabra corta en inglés a su secuencia de valores ASCII (uno por letra), y a la inversa, decodifica una secuencia de valores que se te dé. Después, responde: ¿por qué la palabra "Ñandú" no se puede representar completa usando solo ASCII, y sí usando Unicode?

Actividad (UTF-8): siguiendo el mismo método del ejemplo resuelto de 'ñ' (U+00F1 → `C3 B1`), codifica en UTF-8 el carácter 'é' (*code point* U+00E9, decimal 233), mostrando cada paso: rango del *code point*, número de bytes necesarios, conversión a binario y reparto en el patrón de bits.

Ejemplo resuelto (primer paso, para orientarte): 233 está entre 128 y 2047, así que necesita 2 bytes, con el patrón `110xxxxx 10xxxxxx`.

Completa tú el resto: la conversión de 233 a binario en 11 bits, el reparto en los dos bytes, y el resultado final en hexadecimal.

Actividad (hexadecimal): convierte a hexadecimal los bytes (decimal) 58 y 175. Después, convierte a decimal los valores hexadecimales `4F` y `E8`.

Ejemplo resuelto: 201 → `11001001` → grupos `1100` y `1001` → **C9**.

Actividad (unidades de medida): calcula, mostrando el cálculo, cuántos GiB mostrará aproximadamente el sistema operativo para un disco anunciado como de 256 GB y para uno de 1 TB.

Ejemplo resuelto (256 GB): 256×10^9 bytes $\div 1024^3 = 256.000.000.000 \div 1.073.741.824 \approx$ 238,4 GiB.

Calcula tú el caso del disco de 1 TB.

Actividad (binario/octal): convierte a binario los siguientes valores decimales: 5, 12, 27, 64, 200. Después, agrupa cada resultado de 3 en 3 bits (desde la derecha) para obtener su equivalente en octal.

Actividad (permisos): dados estos permisos en notación simbólica — `rwxr-xr-x`, `rw-rw-r--`, `rwx------` —, obtén su notación octal. Y a la inversa, dados los valores octales 644 y 600, obtén la notación simbólica y explica qué puede hacer cada tipo de usuario.

Ejemplo resuelto: `rwxr-xr-x` → 755.

Resuelve el resto por tu cuenta, aplicando el mismo método (no memorices resultados: descompón cada conjunto en sus tres bits).

6. Sistemas transaccionales y tipos de sistemas de archivos

El problema que resuelven los sistemas transaccionales

Imagina que el ordenador se queda sin electricidad justo mientras se está escribiendo un archivo grande en el disco. Sin ningún mecanismo de protección, el sistema de archivos podría quedar

en un estado inconsistente —corrupto—, con información a medio escribir y sin forma clara de saber qué se llegó a guardar y qué no.

Journaling: cómo se resuelve

Los sistemas de archivos modernos, como **ext4** (habitual en Linux) y **NTFS** (usado por Windows), incluyen un mecanismo llamado **journaling** (o "sistema transaccional"): antes de realizar una operación de escritura, el sistema la registra en un diario (*journal*). Si la operación se interrumpe, al reiniciar el sistema puede consultar ese diario y completar o deshacer la operación de forma limpia, en vez de dejar el sistema de archivos en un estado inconsistente.

Es una de las razones por las que los sistemas operativos actuales rara vez "se rompen" tras un corte de luz o un apagado brusco, a diferencia de sistemas de archivos más antiguos sin esta protección (como FAT32).

Importante: el journaling protege la **consistencia del sistema de archivos**, no evita perder el contenido concreto que se estuviera escribiendo justo en el momento del corte, y **no sustituye a hacer copias de seguridad**. Son dos protecciones distintas que hacen falta las dos.

Panorama de tipos de sistemas de archivos actuales

FAT32, NTFS, ext4 y HFS+/APFS no son "el mismo sistema con distinto nombre": cada uno resuelve el mismo problema —organizar y localizar datos en un disco— con decisiones de diseño distintas, y esas decisiones tienen consecuencias muy prácticas.

Sistema de archivos	SO donde es nativo	Journaling	Permisos tipo Unix (rwx)	Sensible a mayúsculas	Tamaño máx. de archivo (orden de magnitud)	Uso típico hoy
FAT32	Ninguno (el más universal)	No	No	No	4 GB	Memorias USB, tarjetas SD — compatibilidad entre cualquier sistema
NTFS	Windows	Sí	No (modelo propio de ACL, más granular)	No	Del orden de TB	Discos internos de equipos Windows
ext4	Linux	Sí	Sí (nativos)	Sí	Del orden de TB	Discos internos de equipos Linux
HFS+ / APFS	macOS	Sí (HFS+) / copy-on-write (APFS)	Sí	No por defecto	Varios EB	Discos internos de equipos macOS

Los tamaños máximos exactos varían según la implementación concreta — lo relevante no es memorizarlos con precisión de byte, sino entender **por qué existe esa diferencia**: FAT32 es un diseño de los años 90 pensado para tarjetas de memoria pequeñas; los sistemas modernos se diseñaron ya pensando en discos mucho mayores.

Dos columnas de esta tabla conectan con contenido que ya has visto en esta unidad:

- **Permisos tipo Unix**: es la explicación técnica de por qué un archivo puede "perder" sus permisos `rwx` al copiarlo a través de un sistema de archivos que no los soporta (FAT32/exFAT) — como en el caso práctico de cierre de esta unidad.
- **Sensible a mayúsculas** (apartado 4): no es una propiedad de "Windows" o "Linux" en abstracto, es una propiedad del propio sistema de archivos.

VAMOS A PRACTICAR: COMPRUEBA EL SISTEMA DE ARCHIVOS DE UN PENDRIVE REAL

Conecta a tu equipo una memoria USB o disco externo que ya tengas (o pide uno prestado). En Windows, clic derecho sobre la unidad → Propiedades, y localiza el campo "Sistema de archivos". En Linux, el mismo dato aparece con el comando `lsblk -f` en una terminal, o en las propiedades del dispositivo desde el gestor de archivos gráfico.

Anota qué sistema de archivos tiene formateado y compáralo con la tabla de este apartado: ¿tiene sentido ese sistema de archivos para el uso que le das habitualmente a ese dispositivo? Si te sorprende el resultado, piensa si ese pendrive se ha usado alguna vez en un dispositivo distinto a un ordenador (una cámara, una televisión, una videoconsola...), y si eso explica la elección.

Para practicar

Actividad (journaling): explica con tus propias palabras la diferencia entre que un sistema de archivos tenga journaling y que se hagan copias de seguridad periódicas, y por qué hacen falta las dos cosas.

Actividad (elegir sistema de archivos): si vas a formatear un pendrive que debe poder leerse tanto en un Windows como en un Mac como en una cámara de fotos, ¿qué sistema de archivos elegirías y por qué? ¿Y si ese mismo pendrive fuera a usarse solo para hacer copias de seguridad de una carpeta con scripts ejecutables de Linux?

Pista: piensa qué requisito pesa más en cada caso — compatibilidad universal, o conservar permisos de ejecución — y qué fila de la tabla cumple ese requisito.

Caso práctico integrador

Un compañero de prácticas en la empresa te cuenta que ha copiado una carpeta de un proyecto desde un Linux a un pendrive formateado en Windows, y al volver a copiarla a un Linux distinto, los permisos "ya no eran los mismos": algunos scripts que antes eran ejecutables ya no lo son.

Con lo aprendido en esta unidad:

1. Explica qué sistema de archivos maneja permisos Unix de forma nativa y cuál no, y por qué eso puede explicar la pérdida de permisos al pasar por un pendrive.
2. Explica, en términos de binario/octal, qué significa que un script "deje de ser ejecutable" (qué bit ha cambiado).

3. Relaciona el caso con la arquitectura por capas: ¿en qué capa se decide qué permisos tiene un archivo?

Resumen de la unidad

- Un sistema informático es hardware + software + usuarios; el software se divide en software de base y software de aplicación.
- El sistema operativo se organiza por capas (núcleo, controladores, shell, aplicaciones) y cumple cuatro funciones: gestión de procesos, memoria, archivos y entrada/salida.
- Windows 11 y Linux gestionan la memoria mediante paginación por demanda: solo cargan en RAM las páginas que se usan en cada momento, recurriendo al disco (paginación/swap) cuando hace falta más sitio.
- La interfaz gráfica es la versión gráfica del shell, no una capa aparte ni el sistema operativo en sí.
- La CPU trabaja en modo núcleo (todo permitido: núcleo y controladores) o en modo usuario (aplicaciones, con acceso restringido). Las aplicaciones piden los servicios al núcleo mediante llamadas al sistema. Ser administrador o `root` no significa ejecutarse en modo núcleo.
- Los núcleos pueden ser monolíticos (Linux: todo en modo núcleo), microkernel (solo lo mínimo en modo núcleo) o híbridos (Windows, macOS: diseño por piezas, pero casi todo en modo núcleo) — son diseños distintos, no una jerarquía de calidad.
- Un proceso es un programa en ejecución, con estados: nuevo, listo, ejecución, bloqueado, terminado.
- Windows organiza los archivos con letras de unidad; Linux usa una única raíz `/` y distingue mayúsculas de minúsculas.
- El texto también se representa en binario: Unicode asigna un número (*code point*) a cada carácter posible; UTF-8 (la codificación dominante hoy) convierte ese número en 1 a 4 bytes según un patrón de bits fijo, manteniendo compatibilidad con ASCII; UTF-16 sigue viva por dentro de Windows, Java y JavaScript.
- El hexadecimal agrupa el binario de 4 en 4 bits: un byte se escribe siempre con 2 dígitos hex, por eso se usa para colores web, direcciones MAC y los bytes de UTF-8.
- El byte tiene múltiplos según dos normas: SI (base 10, el que anuncia el fabricante) e IEC 80000-13 (base 2, el que suele calcular y mal-etiquetar el sistema operativo) — de ahí que un disco de 500 GB se muestre como ~465 GB.
- Un dígito octal representa un conjunto completo de permisos `rwx` (3 bits). Los permisos se definen para propietario, grupo y otros.
- El journaling protege la consistencia del sistema de archivos ante interrupciones, pero no sustituye a las copias de seguridad.
- FAT32 (universal, sin journaling ni permisos Unix), NTFS (Windows), ext4 (Linux) y HFS+/APFS (macOS) son sistemas de archivos distintos, no intercambiables.

Relación con RA y CE

CE	Contenido	Dónde se trabaja
a)	Elementos funcionales de un sistema informático	Apartado 1
b)	Representación de la información	Apartado 5
c)	Funciones del sistema operativo	Apartado 2
d)	Arquitectura del sistema operativo	Apartado 2
e)	Procesos y sus estados	Apartado 3
f)	Estructura y organización del sistema de archivos	Apartado 4
g)	Atributos de archivo y directorio	Apartado 4
h)	Permisos de archivos y directorios	Apartado 5
i)	Sistemas transaccionales	Apartado 6